

Project 1: Socket Programming — Protocol Design

ชื่อโปรแกรม/Protocol: SFTP-Lite ผู้พัฒนา: Chissanapun Athiwarikanon รายวิชา: CS351 Computer Communications

1. วัตถุประสงค์ของโปรแกรม (Objective)

SFTP-Lite เป็น network application สำหรับโอนไฟล์ระหว่างอุปกรณ์ของผู้ใช้เองในวง LAN เดียวกัน (เช่น โน้ตบุ๊ก <-> เครื่องในห้องแล็บ <-> มือถือผ่าน Termux) โดยไม่ต้องผ่าน cloud service

ปัญหาที่แก้: การส่งไฟล์ผ่านแอปแชท (LINE, Messenger) ให้ตัวเองมักถูกบีบอัดคุณภาพ (รูป/วิดีโอ) โดยอัตโนมัติ และต้องพึ่งอินเทอร์เน็ตแม้อุปกรณ์ทั้งสองฝั่งจะอยู่ห้องเดียวกัน SFTP-Lite ส่งไฟล์แบบ raw bytes ตรงระหว่างเครื่องในวง WiFi/LAN เดียวกัน ไม่บีบอัด ไม่ต้องเน็ต ไม่ต้องเสียบสาย (แค่อยู่ network เดียวกัน)

2. คุณลักษณะของแอปพลิเคชัน (Application Characteristics)

โปรแกรมประกอบด้วย 2 ส่วนที่ทำงานร่วมกัน:

1. **ระบบหลัก — File Transfer** (STORE / GET / LIST / DELETE): client-server, client ร้องขอ server ตอบ, มีการโอนข้อมูล (ไฟล์) ปริมาณตั้งแต่ไม่กี่ byte ถึงหลาย MB
2. **ระบบรอง — Discovery Beacon**: server ประกาศตัวตนเป็นระยะบนวง LAN เพื่อให้ client หา IP ของ server เจอโดยไม่ต้องรู้ IP ล่วงหน้า (แบบ AirDrop)

ทั้งสองระบบมีลักษณะงานต่างกัน จึงพิจารณา transport service model แยกกัน (รายละเอียดข้อ 3)

3. Transport Layer Service Model ที่ต้องการ — วิเคราะห์ทั้ง 2 ระบบแยกกัน

อ้างอิงตาราง 4 มิติที่ใช้พิจารณา transport service (data integrity, timing, throughput, security) จาก Ch2 §3

3.1 ระบบหลัก (File Transfer) → TCP

มิติ	ความต้องการของ SFTP-Lite (file transfer)	บังคับหรือไม่
Data integrity	ไฟล์ต้องไม่หาย/ไม่เพี้ยน — byte ผิดแม้จุดเดียวทำให้ไฟล์เสียหายทั้งไฟล์ (เช่น ZIP/PDF ที่ header ผิดเปิดไม่ได้เลย)	บังคับ → ต้องการ reliable, in-order delivery
Timing	ไม่ time-sensitive เหมือน real-time audio/video การหน่วงเป็นวินาทีไม่กระทบผลลัพธ์	ไม่บังคับทางใดทางหนึ่ง
Throughput	Elastic — ทำงานได้ทั้งบน WiFi ช้า/เร็ว ไม่ต้องการ bandwidth ขั้นต่ำ	ไม่บังคับทางใดทางหนึ่ง

มิติ	ความต้องการของ SFTP-Lite (file transfer)	บังคับหรือไม่
Security	ไม่มีการเข้ารหัสระดับ protocol (cleartext, ไม่มี authentication) — เป็นการตัดสินใจจำกัดขอบเขตโดยตั้งใจ ไม่ใช่ข้อบกพร่องที่ลึ้มทำ (อยู่ในกลุ่มเดียวกับ HTTP ธรรมดาที่ไม่มี TLS)	ไม่บังคับทางใดทางหนึ่ง

สรุป: มีเพียงมิติ data integrity เท่านั้นที่บังคับทิศทางชัดเจน และบังคับไปทาง **TCP** (reliable, in-order, connection-oriented) ส่วนอีก 3 มิติไม่ได้บังคับไปทาง UDP เช่นกัน — จึงไม่มีเหตุผลใดที่จะเลือก UDP สำหรับงานนี้

3.2 ระบบรอง (Discovery Beacon) → UDP

มิติ	ความต้องการของ discovery beacon	บังคับหรือไม่
Data integrity	ถ้า beacon 1 ครั้งหาย ไม่กระทบระบบ — server ส่งซ้ำทุก 2 วินาทีอยู่แล้ว client แคร่รอรอบถัดไป	ไม่บังคับ → ทนต่อการสูญหายได้ (loss-tolerant)
Timing/Throughput	ไม่มีข้อกำหนด	ไม่บังคับ
โครงสร้างการสื่อสาร	server ต้องประกาศไปยังทุกเครื่องในวง LAN พร้อมกันโดยไม่รู้ล่วงหน้าว่ามี client ก็เครื่อง/อยู่ที่ไหน — เป็นลักษณะ one-to-many broadcast	บังคับ → TCP ทำไม่ได้ (TCP เป็น point-to-point ต้อง connect() ไปยังปลายทางที่รู้ IP ล่วงหน้าเท่านั้น ตามที่เรียนใน Ch3)

สรุป: งาน discovery ไม่ต้องการความน่าเชื่อถือ (ส่งซ้ำเป็นระยะอยู่แล้ว) และต้องการ broadcast แบบ one-to-many ซึ่งเป็นสิ่งที่ TCP ทำไม่ได้โดยธรรมชาติของ protocol เอง จึงเลือก **UDP**

3.3 สรุปรวม

SFTP-Lite ไม่ได้ “เลือกไม่ถูกระหว่าง TCP กับ UDP” แต่เป็นแอปที่มี **2 งานย่อยที่มีลักษณะต่างกันจริง** และแต่ละงานเลือก transport ที่เหมาะกับตัวเองแยกกัน: - โอนไฟล์ (ต้องการความถูกต้อง, point-to-point) → **TCP** - ประกาศตัวตน (ทนการสูญหายได้, ต้อง broadcast) → **UDP**

4. Application-Layer Protocol Design

4.1 ภาพรวม session (โครงสร้างแบบ SMTP: handshake → transfer → closure)

[client เชื่อมต่อ TCP]

S: 220 SFTP-Lite ready\r\n

C: STORE <filename> <size_bytes>\r\n

S: 100 Continue\r\n
 หรือ 400 Bad Request\r\n (ชื่อไฟล์ผิด/size ไม่ใช่ตัวเลข — client **ไม่ส่ง** payload ถ้าเจอ response นี้)
 C: <raw file bytes, ความยาวตรงตาม size_bytes — ส่งเฉพาะเมื่อได้ 100 Continue เท่านั้น>
 S: 201 Stored <size_bytes> bytes\r\n
 หรือ 500 Server Error (เขียนไฟล์ไม่สำเร็จ)

C: GET <filename>\r\n
 S: 200 OK <size_bytes>\r\n
 <raw file bytes>
 หรือ 404 Not Found (ไม่พบไฟล์)

C: LIST\r\n
 S: 200 OK <count>\r\n
 <รายชื่อไฟล์ count บรรทัด>
 (count = 0 ได้ ถ้าไม่มีไฟล์)

C: DELETE <filename>\r\n
 S: 200 OK Deleted\r\n
 หรือ 404 Not Found

C: QUIT\r\n
 S: 221 Closing\r\n
 [server ปิด connection]

คำสั่งที่ไม่รู้จัก/รูปแบบผิด (ทุกกรณี) → 400 Bad Request

4.2 ตาราง Request/Response Message (syntax + semantics)

Request	Syntax	Semantics
STORE <filename> <size>	ASCII command line จบด้วย \r\n ตามด้วย raw bytes ยาว <size> byte	ขอเก็บไฟล์ชื่อ <filename> ขนาด <size> byte ไว้บน server
GET <filename>	ASCII command line จบด้วย \r\n	ขอไฟล์ <filename> คืนจาก server
LIST	ASCII command line จบด้วย \r\n, ไม่มีพารามิเตอร์	ขอรายชื่อไฟล์ทั้งหมดบน server
DELETE <filename>	ASCII command line จบด้วย \r\n	ขอลบไฟล์ <filename> บน server
QUIT	ASCII command line จบด้วย \r\n	ขอปิดการเชื่อมต่อ

Response	Status	Phrase	ความหมาย
Greeting	220	Ready	server พร้อมรับคำสั่ง (ส่งทันทีหลัง TCP handshake เสร็จ)
Continue	100	Continue	server ตรวจ STORE header ผ่านแล้ว พร้อมรับ payload — client ส่ง raw bytes ได้
Success (store)	201	Stored	บันทึกไฟล์สำเร็จ
Success (generic)	200	OK	คำสั่งสำเร็จ (GET/LIST/DELETE)

Response	Status	Phrase	ความหมาย
Client error	400	Bad Request	คำสั่งผิดรูปแบบ, ชื่อไฟล์ไม่ถูกต้อง (ดู path traversal guard)
Not found	404	Not Found	ไม่พบไฟล์ที่ร้องขอ (GET/DELETE)
Server error	500	Server Error	เขียน/อ่านไฟล์บน server ล้มเหลว
Closing	221	Closing	server ยืนยันปิดการเชื่อมต่อ

4.3 กฎสำคัญของ implementation

TCP เป็น byte stream ไม่มี message boundary (Ch3) — กฎนี้ใช้กับทั้งสองฝั่งและทั้งสองทิศทาง: - STORE: **server** ต้อง loop รับ (recv) สะสมจนครบ <size> byte ที่ client ประกาศไว้ - GET: **client** ต้อง loop รับ (recv) สะสมจนครบ <size> byte ที่ server ตอบกลับมา

ห้ามสมมติว่า recv() ครั้งเดียวจะได้ข้อมูลครบ ไม่ว่าฝั่งไหนเพราะ TCP อาจแบ่งส่งเป็นหลาย segment โดยเฉพาะบนเครือข่ายจริง (ต่างจาก localhost ที่มักไม่แบ่ง)

การแยก command line ออกจาก payload (เส้นแบ่งที่แท้จริงของ byte stream): เนื่องจากไม่มี message boundary, STORE <filename> <size>\r\n กับ raw file bytes ที่ตามมาอยู่ใน stream เดียวกันต่อเนื่องกัน การอ่านด้วย recv(N) แบบไม่ระวังอาจดึงข้อมูลไฟล์บางส่วนติดมากับ command line โดยไม่ตั้งใจ กฎที่ใช้: อ่านทีละ chunk สะสมไว้ใน buffer จนเจอ \r\n ตัวแรก — ส่วนก่อน \r\n คือ command line ส่วนที่เหลือใน buffer (ถ้ามี) คือ byte แรกๆ ของ payload ที่ต้องนับรวมเข้ากับ <size> byte ที่ต้องอ่านต่อ ไม่ใช่ทิ้งไป

LIST termination: แต่ละชื่อไฟล์อยู่คนละบรรทัด จบด้วย \r\n ผู้รับอ่านต่อเนื่องไปที่ละบรรทัดจนครบ <count> บรรทัดตามที่ประกาศไว้ใน response line (ไม่ใช่รอบรรทัดว่างหรือสัญลักษณ์จบพิเศษอื่น)

Partial transfer: ถ้าการเชื่อมต่อขาดกลางทางก่อนรับครบ <size> byte ต้องทิ้งไฟล์ที่รับมาไม่ครบทั้งหมด ไม่เก็บไฟล์ที่เสียหายไว้บน server

ทำไมต้องมี 100 Continue ก่อน payload: ถ้า server ปฏิเสธ STORE (เช่น ชื่อไฟล์ผิด) แล้วตอบ 400 ทันที โดยที่ client ส่ง payload ตามหลัง header ไปแล้วโดยไม่รอคำตอบ — payload นั้นจะยังค้างอยู่ใน TCP stream และถูกตีความผิดเป็นคำสั่งถัดไปในรอบถัดไป (session desync) ทำให้ session พังทั้งหมด การบังคับให้ client รอ 100 Continue ก่อนส่ง payload เสมอ ตัดปัญหานี้ตั้งแต่ระดับ protocol design ไม่ต้องแก้ที่ implementation ที่หลัง

Path traversal guard: ชื่อไฟล์ที่มีเครื่องหมาย / หรือ .. ต้องถูกปฏิเสธด้วย 400 Bad Request เพื่อป้องกันไม่ให้ client เขียน/อ่านไฟล์นอก storage directory ที่กำหนด

4.4 Discovery Sub-protocol (UDP, แยกจากระบบหลัก — ดูข้อ 3.2)

[Server, ทุก 2 วินาที]

Server → broadcast (255.255.255.255, DISCOVERY_PORT):

"SFTPLITE_ANNOUNCE <tcp_port>"

[Client, เมื่อสั่ง `sftplite discover`]

Client bind DISCOVERY_PORT, รอฟัง broadcast สูงสุด N วินาที

เมื่อได้รับ message ที่ขึ้นต้นด้วย "SFTPLITE_ANNOUCE":

แสดง IP ต้นทาง (จาก recvfrom) + tcp_port ที่ประกาศมา

ไม่มี response/acknowledgment กลับจาก client — เป็น one-way broadcast ตามธรรมชาติของ UDP ที่ไม่ต้อง hand-shake

หมายเหตุ implementation: broadcast address (255.255.255.255 หรือ subnet broadcast เช่น 192.168.1.255 ขึ้นกับ network setup จริง) ทำเป็นค่า config ได้ ไม่ hardcode ตายตัว — เพื่อไม่ให้ demo ล้มเหลวเพราะ network จริงต้องการ subnet broadcast แทน ต้องตั้ง SO_BROADCAST บน socket ฝั่งส่ง และ SO_REUSEADDR บน socket ฝั่งรับ (กันปัญหา bind ชนกันถ้ารัน server/client ทดสอบบนเครื่องเดียวกัน)

4.5 พฤติกรรมที่ต้องสังเกตได้ (logging requirement)

ทั้ง client และ server ต้อง **print ทุกบรรทัดของ protocol message ที่ส่ง/รับ พร้อม status code + phrase** โดยระบุทิศทางชัดเจน (เช่น prefix >> สำหรับส่ง, << สำหรับรับ) — นี่คือนิยามที่โจทย์ข้อ 2 ต้องการให้เห็นตอน demo และเป็นสิ่งที่พิสูจน์ว่าโค้ดกับ spec ในเอกสารนี้เป็นสิ่งเดียวกันจริง

4.6 Character encoding และข้อจำกัดของชื่อไฟล์

command line ทั้งหมด (ทั้ง request และ response) เข้ารหัสเป็น **UTF-8** ไม่ใช่ ASCII ล้วนแบบ SMTP ดั้งเดิม — เพราะไฟล์จริงที่ใช้งานอาจมีชื่อภาษาไทย และ UTF-8 ยังคง backward-compatible กับ ASCII สำหรับ keyword/ตัวเลขในโปรโตคอล การใช้ \r\n เป็นตัวแบ่งบรรทัดยังปลอดภัย เพราะ byte \r (0x0D) และ \n (0x0A) ไม่มีทางปรากฏภายใน multi-byte UTF-8 sequence (ทุก continuation/lead byte ของ UTF-8 มีค่า $\geq 0x80$)

ข้อจำกัดที่ทราบ (known limitation): ชื่อไฟล์ที่มี “ช่องว่าง” ยังไม่รองรับ เพราะ parser แยก command ด้วย space เป็นตัวคั่น (STORE <filename> <size>) — ถ้าชื่อไฟล์มีช่องว่างจะถูกตีความผิดและได้ 400 Bad Request กลับมา (fail safely, ไม่ crash) ไม่อยู่ในขอบเขตที่ต้องแก้สำหรับ submission นี้

5. สรุป

SFTP-Lite ใช้ TCP เป็น transport หลักสำหรับงานที่ต้องการความถูกต้องของข้อมูล (โอนไฟล์) และใช้ UDP สำหรับงานเสริมที่ต้องการ broadcast และทนต่อการสูญหายได้ (discovery) — เป็นการเลือก transport ที่พิจารณาจากลักษณะงานจริงของแต่ละ ไม่ใช้การเลือกแบบเดียวสำหรับทั้งโปรแกรม